NED University of Engineering and Technology
Department of Computer Science and Information Technology

CT-463 DATA WAREHOUSE & MINING

CCP REPORT

Retail Customer Analytics Data Warehouse

INSTRUCTOR: Ms. Mehar Fatima

Group Number: 9 **Batch: 2022-26**

Group Members

Laibah Shahid	CT–22061
Tooba Kashaf	CT–22068
Haiqa Siddiqua	CT–22071
Muhammad Zohaib Khan	CT–22072

S.NO	CONTENTS	PG_NO

Acknowledgement

We would like to express our sincere gratitude to our teacher, Ms. ***Mehar Fatima***, for his invaluable guidance and support throughout the project. His expertise and commitment have been a great source of inspiration for us.

Executive Summary

The *E-commerce Retail Data Warehouse Project* aims to design and implement a centralized analytical system for online retail operations. Using a UK-based e-commerce dataset as the primary source, transactional, customer, and product data were extracted, transformed, and loaded (ETL) into **BigQuery** to support fast, reliable, and scalable business insights. The data warehouse integrates information from **multiple heterogeneous sources**, including three OLTP systems (sales, customers, and products) and several external flat files simulating API feeds such as **exchange rates, holiday calendars, customer feedback, website traffic, and shipping data**.

The data was modeled using a **Star Schema** architecture comprising multiple dimension tables (Customer, Product, Holiday, Shipping, Traffic, and Exchange Rate) linked to a central **FactSales** table. Each dataset was cleaned, validated, and enriched using Python and Faker to ensure data accuracy, consistency, and completeness. The design emphasizes scalability, automation, and flexibility for future analytical extensions, including sales forecasting, customer segmentation, and anomaly detection.

Through this implementation, the warehouse enables comprehensive performance tracking such as total sales, customer engagement, top-performing products, and region-wise revenue trends. The BI-ready model supports visualization in Power BI, providing interactive dashboards for real-time, data-driven decision-making. Overall, the project demonstrates the complete data warehousing lifecycle, from multi-source integration and ETL to analytical modeling, validation, and visualization.

1. INTRODUCTION

In today's digital economy, the e-commerce industry generates massive volumes of data every second through online transactions, customer interactions, and digital marketing activities. Managing this data efficiently is crucial for understanding consumer behavior, optimizing operations, and supporting strategic decision-making. However, the data collected from multiple sources, such as sales systems, product catalogs, customer profiles, and third-party APIs,often exists in disparate formats, making it difficult to derive unified insights. To address these challenges, this project focuses on developing a centralized **E-commerce Retail Data Warehouse** designed to integrate, cleanse, and organize retail data for analytical and business intelligence purposes.

The proposed system leverages **Google BigQuery** as the core warehousing platform, incorporating diverse data sources including structured tables (*Sales*, *Products*, *Customers*), flat files (*Feedback*, *Holidays*, *Shipping*), and API-driven datasets (*Website Traffic*, *Exchange Rate*). Through a well-defined **ETL pipeline** and **Star Schema** architecture, the system supports efficient data querying, visualization, and reporting. Furthermore, **Power BI** is used to generate Key Performance Indicators (KPIs) and interactive dashboards, while **Machine Learning, Deep Learning, and Data Mining** models enhance decision support by providing predictive and automated insights. The overall objective is to transform raw, scattered data into actionable intelligence that drives performance improvement and business growth in the e-commerce domain.

2. OVERVIEW OF THE SYSTEM

The proposed system functions as a unified **E-commerce Retail Data Platform**, integrating data ingestion, warehousing, analytics, and AI-driven insights. It is designed to consolidate diverse retail data sources to support performance monitoring, business intelligence, and predictive analytics.

2.1 Data Sources:

The system ingests data from multiple heterogeneous sources, including **transactional datasets** such as *Sales*, *Products*, and *Customers* tables stored in BigQuery, as well as **flat files** like *Customer Feedback*, *Holiday Calendar*, and *Shipping Details* in CSV format. Additionally, **API-based datasets** such as *Website Traffic* and *Exchange Rate* are integrated to enhance analytical depth by correlating sales with market trends and online engagement.

2.2 Data Integration:

An automated **ETL (Extract, Transform, Load)** pipeline is developed to clean, standardize, and harmonize data from all sources. The data is structured into a **Star Schema**, consisting of a central *FactSales* table linked with multiple dimension tables (*DimCustomer*, *DimProduct*, *DimDate*, *DimShipping*, *DimFeedback*, etc.). This schema design optimizes query performance and facilitates multidimensional analysis across sales, customer behavior, and operational metrics.

2.3 Analytics & Visualization:

The processed and aggregated data is connected to **Power BI** for KPI measurement, visualization, and reporting. Interactive dashboards display metrics such as *Total Sales*

Revenue, *Customer Retention Rate*, *Product Category Performance*, and *Shipping Efficiency*. These dashboards enable stakeholders to monitor key performance indicators (KPIs) and make data-driven decisions in real time.

2.4 Machine Learning & Data Mining:

A layer of **Machine Learning and Data Mining** is integrated with the warehouse for advanced analytics. Predictive models are trained to forecast *sales trends*, *customer churn probability*, and *demand fluctuations*. Deep learning techniques are applied to analyze *customer feedback sentiment*, while data mining tools assist in generating *segment customers*. The models are periodically retrained using warehouse data, ensuring continuous improvement and integration within the analytics pipeline.

3. FUNCTIONAL FEATURES

3.1 Data Extraction & Ingestion

● **Transactional Data:** Extracted from core relational tables including *Sales*, *Products*, and *Customers* within the e-commerce database.
● **Flat Files:** Supplementary data such as *Customer Feedback*, *Holiday Calendar*, and *Shipping Details* imported as CSV files.
● **API Sources:** Real-time data from *Website Traffic* and *Exchange Rate* APIs integrated to capture external business dynamics and market conditions.

3.2 Data Transformation & Cleaning

● Standardization of inconsistent formats such as *dates*, *currency*, and *product category labels* across all sources.
● Handling of *missing values*, removal of *duplicates*, and normalization of categorical variables to maintain uniformity.
● Derivation of analytical attributes such as *profit margin*, *discount percentage*, *customer tenure*, and *holiday impact factor* to enhance analysis.

3.3 Data Warehouse Design (Star Schema)

● **Fact Table:** *FactSales* – stores transactional metrics like total sales amount, quantity sold, discount, and profit.
● **Dimension Tables:** *DimCustomer*, *DimProduct*, *DimDate*, *DimShipping*, *DimFeedback*, *DimExchangeRate*, and *DimWebsiteTraffic*.
● The Star Schema ensures query optimization, supports OLAP operations, and simplifies KPI computation across multiple dimensions.

3.4 ETL Pipeline Implementation

● Automated **ETL workflows** developed in BigQuery to extract, clean, and load data from all sources into the warehouse.
● Data transformation scripts executed to join heterogeneous datasets, enforce referential integrity, and populate fact-dimension relationships.
● Periodic scheduling, logging, and data quality validation implemented to ensure consistency and reliability of loaded datasets.

3.5 KPI Generation & Visualization

● **Key KPIs:**

- Total and Average Sales Revenue
- Top-Selling Products and Categories
- Customer Retention & Acquisition Rate
- Average Order Value (AOV) and Conversion Rate
- Shipping Delay Rate and Feedback Sentiment Score
- Sales vs. Exchange Rate and Holiday Impact Analysis

● **Visualization Platform:** Dashboards built in **Power BI** provide real-time insights

for business managers, enabling sales performance tracking, customer segmentation, and profitability analysis.

3.6 Machine Learning & AI Features

The AI and ML layer adds predictive, diagnostic, and generative intelligence to the retail analytics system.

1. **Sales Forecasting (Time Series Regression)**
   ○ Predicts future sales trends using historical data, seasonal patterns, and holiday influences.
   ○ Assists in demand planning and inventory optimization.
2. **Customer Segmentation (K-Means Clustering)**
   ○ Groups customers based on purchase frequency, average order value, and feedback sentiment.
   ○ Helps identify loyal, at-risk, and high-value customer segments.
3. **Sentiment Analysis (Deep Learning Model)**
   ○ Analyzes customer feedback text to derive sentiment polarity (positive, neutral, negative).
   ○ Provides insight into service satisfaction and product quality perception.
4. **Generative AI Insights (GenAI Integration)**
   ○ Uses large language models to automatically summarize analytics dashboards and generate strategic insights.
   ○ Enables natural language queries for decision support and executive summaries.

4. Data Modeling:

5. Foreign Keys

1. **dim_customer table:**
   ○ region_id: References dim_region.region_id (One-to-Many relation with *dim_region*).
   ○ loyalty_tier_id: References dim_loyalty_tier.loyalty_tier_id (One-to-Many relation with *dim_loyalty_tier*).
2. **dim_product table:**
   ○ category_id: References dim_category.category_id (One-to-Many relation with *dim_category*).
   ○ supplier_id: References dim_supplier.supplier_id (One-to-Many relation with *dim_supplier*).
3. **dim_shipping table:**
   ○ carrier_id: References dim_carrier.carrier_id (One-to-Many relation with *dim_carrier*).
   ○ region_id: References dim_region.region_id (One-to-Many relation with *dim_region*).
4. **dim_feedback table:**
   ○ customer_id: References dim_customer.customer_id (One-to-Many relation with *dim_customer*).
   ○ product_id: References dim_product.product_id (One-to-Many relation with *dim_product*).
5. **dim_exchange_rate table:**
   ○ currency_id: References dim_currency.currency_id (One-to-Many relation with *dim_currency*).
6. **dim_website_traffic table:**
   ○ date_id: References dim_date.date_id (One-to-Many relation with *dim_date*).
7. **fact_sales table:**
   customer_id: References dim_customer.customer_id (One-to-Many relation with *dim_customer*).
   ○ product_id: References dim_product.product_id (One-to-Many relation with *dim_product*).
   ○ shipping_id: References dim_shipping.shipping_id (One-to-Many relation with *dim_shipping*).
   ○ feedback_id: References dim_feedback.feedback_id (One-to-Many relation with *dim_feedback*).
   ○ exchange_rate_id: References dim_exchange_rate.exchange_rate_id (One-to-Many relation with *dim_exchange_rate*).
   ○ traffic_id: References dim_website_traffic.traffic_id (One-to-Many relation with *dim_website_traffic*).
   ○ date_id: References dim_date.date_id (One-to-Many relation with *dim_date*).

6. ETL Pipeline Development

The **ETL (Extract, Transform, Load)** pipeline forms the backbone of the E-commerce Retail Data Warehouse, orchestrating the collection, cleaning, transformation, and consolidation of data from multiple heterogeneous sources into a unified analytical structure. This pipeline ensures that the warehouse contains high-quality, consistent, and analysis-ready data.

6.1 Extraction Phase

● **Transactional Sources:** Data is extracted from core relational tables including *Sales*, *Products*, and *Customers* stored in BigQuery.
● **Flat File Sources:** Supplementary information such as *Customer Feedback*, *Holiday Calendar*, and *Shipping Details* is extracted from CSV files using Python's `pandas` library.
● **API Sources:** Real-time datasets like *Website Traffic* and *Exchange Rates* are retrieved via API calls using Python's `requests` library, ensuring the warehouse captures external business dynamics.
● The extraction process is automated, handling incremental loads and full refreshes to keep the warehouse updated.

6.2 Transformation Phase

● Raw data from all sources is cleaned and standardized to ensure uniformity across datasets.
● Key transformation operations include:
● Handling missing or inconsistent values (e.g., replacing null sales amounts with 0 or average order value).
● Converting data types appropriately (e.g., string dates to `datetime` format, numeric currencies to standardized format).
● Normalization and encoding of categorical fields (e.g., product categories, customer segments).
● Creating derived features such as *profit margin*, *discount percentage*, *customer tenure*, *holiday flag*, and *feedback sentiment score*.
● The transformation process aligns the data with the **Star Schema** structure of the warehouse, ensuring all fact and dimension tables are populated correctly.

6.3 Loading Phase

● The cleaned and transformed data is loaded into BigQuery fact and dimension tables.
● **Fact Table:** *FactSales* stores quantitative metrics such as sales amount, quantity sold, discounts, profit, and website traffic metrics.
● **Dimension Tables:** *DimCustomer*, *DimProduct*, *DimDate*, *DimShipping*, *DimFeedback*, *DimExchangeRate*, and *DimWebsiteTraffic* describe the entities, providing context for analytical queries.
● Referential integrity is maintained throughout the loading process by enforcing primary–foreign key relationships, ensuring accurate joins between facts and dimensions.
● The ETL pipeline is designed for scheduling, logging, and monitoring, providing a reliable and reproducible process for data integration.

Extraction and Transformation:

enrich_customers.py > ...

```python
# enrich_customers.py
import pandas as pd
from faker import Faker
import random

fake = Faker()
random.seed(42)

# 1. Load your downloaded customers CSV
df = pd.read_csv("raw_customers.csv")

# 2. Enrich with realistic data
def random_loyalty():
    return random.choices(['Gold', 'Silver', 'Bronze'], weights=[0.1, 0.3, 0.6])[0]

n = len(df)
df['CustomerName'] = [fake.name() for _ in range(n)]
df['Email'] = [fake.email() for _ in range(n)]
df['Gender'] = [random.choice(['M', 'F']) for _ in range(n)]
df['LoyaltyLevel'] = [random_loyalty() for _ in range(n)]
df['SignupDate'] = [fake.date_between(start_date='-5y', end_date='today') for _ in range(n)]
df['City'] = [fake.city() for _ in range(n)]

# 3. Save enriched version
df.to_csv("raw_customers_enriched.csv", index=False)

print("✅ Enriched customer data saved as raw_customers_enriched.csv")

```

enrich_products.py > ...

```python
# enrich_products.py
import pandas as pd
from faker import Faker
import random
import re

fake = Faker()
random.seed(123)

# 1. Load your downloaded products CSV
df = pd.read_csv("raw_products.csv")

# 2. Define category mapping
def assign_category(desc):
    d = (str(desc) or "").upper()
    if re.search(r'MUG|CUP|TEA|COFFEE', d):
        return 'Kitchen'
    if re.search(r'LAMP|LANTERN|LIGHT|BULB', d):
        return 'Lighting'
    if re.search(r'HEART|COAT|HANGER|CUSHION|HOME', d):
        return 'Home Decor'
    if re.search(r'SOCK|SCARF|HAT|GLOVE', d):
        return 'Apparel'
    if re.search(r'BATTERY|USB|CHARGER|ELECTR', d):
        return 'Electronics'
    return 'Misc'

# 3. Enrich
df['Category'] = df['Description'].apply(assign_category)
df['Supplier'] = [fake.company() for _ in range(len(df))]
df['Brand'] = [fake.word().title() for _ in range(len(df))]
df['LaunchDate'] = [fake.date_between(start_date='-3y', end_date='today') for _ in range(len(df))]
df['ReorderLevel'] = [random.randint(10, 200) for _ in range(len(df))]

# 4. Save enriched version
df.to_csv("raw_products_enriched.csv", index=False)
```

```python
# shipping_generator.py
import pandas as pd
import random
from faker import Faker
from datetime import timedelta

# Initialize Faker and random
fake = Faker()
random.seed(42)

# Load your sales CSV (adjust filename if different)
sales = pd.read_csv("raw_sales.csv", encoding="latin1")

# Select only InvoiceNo and InvoiceDate
sales = sales[['InvoiceNo', 'InvoiceDate']].dropna().drop_duplicates()

# Convert InvoiceDate to datetime
sales['InvoiceDate'] = pd.to_datetime(sales['InvoiceDate'], errors='coerce')
sales = sales.dropna(subset=['InvoiceDate'])

# Limit to smaller sample if needed
```

```
sales = sales.head(5000)  # adjust this number as you like

couriers = ["Royal Mail", "DHL", "FedEx", "UPS", "Hermes"]
statuses = ["Delivered", "In Transit", "Returned", "Cancelled"]

rows = []

for _, r in sales.iterrows():
    invoice = r['InvoiceNo']
    ship_date = r['InvoiceDate'] + pd.to_timedelta(random.randint(0, 2),
unit='D')
    delivery_offset = random.choice([1, 2, 3, 4, 5, 6, 7])
    delivery = ship_date + pd.to_timedelta(delivery_offset, unit='D')
    courier = random.choice(couriers)
    cost = round(max(0.0, random.gauss(5.0, 2.0)), 2)
    status = "Delivered" if delivery <= pd.Timestamp.today() else
random.choice(statuses)

    rows.append({
        "InvoiceNo": invoice,
        "ShipmentDate": ship_date.date(),
        "DeliveryDate": delivery.date(),
        "Courier": courier,
        "ShippingCost": cost,
        "Status": status
    })

# Create DataFrame and save
shipping_df = pd.DataFrame(rows)
shipping_df.to_csv("shipping_data.csv", index=False)
```

traffic_generator_local.py > ...

```python
# traffic_generator_local.py
import pandas as pd
import random
from faker import Faker

fake = Faker()
random.seed(123)

# Load your sales CSV
sales = pd.read_csv("raw_sales.csv", encoding="latin1")

# Extract distinct dates and countries
sales['InvoiceDate'] = pd.to_datetime(sales['InvoiceDate'], errors='coerce')
sales = sales.dropna(subset=['InvoiceDate', 'Country'])

dates_countries = sales[['InvoiceDate', 'Country']].drop_duplicates()

rows = []
for _, r in dates_countries.iterrows():
    date = r['InvoiceDate'].date()
    country = r['Country']

    pageviews = int(max(100, random.gauss(1500, 800)))
    unique_visitors = int(pageviews * random.uniform(0.4, 0.8))
    avg_session_duration = round(random.uniform(1.0, 4.5), 2)

    rows.append({
        "Date": date,
        "Country": country,
        "PageViews": pageviews,
        "UniqueVisitors": unique_visitors,
        "AvgSessionDuration": avg_session_duration
    })

traffic_df = pd.DataFrame(rows)
traffic_df.to_csv("website_traffic.csv", index=False)
```

Sales, Product, Customer Deduplication

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_staging.sales_deduped` AS
SELECT
  InvoiceNo,
  StockCode,

  -- Keep one CustomerID per invoice-product combo
  ARRAY_AGG(CustomerID LIMIT 1)[OFFSET(0)] AS CustomerID,

  -- Aggregate metrics
  SUM(Quantity) AS Quantity,
  ARRAY_AGG(UnitPrice LIMIT 1)[OFFSET(0)] AS UnitPrice,
  SUM(TotalAmount) AS TotalAmount,

  -- Retain dimension mapping attributes
  ARRAY_AGG(Country LIMIT 1)[OFFSET(0)] AS Country,
  ARRAY_AGG(HolidayName LIMIT 1)[OFFSET(0)] AS HolidayName,
  ARRAY_AGG(CustomerKey LIMIT 1)[OFFSET(0)] AS CustomerKey,
  ARRAY_AGG(ProductKey LIMIT 1)[OFFSET(0)] AS ProductKey,
  ARRAY_AGG(HolidayKey LIMIT 1)[OFFSET(0)] AS HolidayKey,
  ARRAY_AGG(ExchangeKey LIMIT 1)[OFFSET(0)] AS ExchangeKey,
  ARRAY_AGG(TrafficKey LIMIT 1)[OFFSET(0)] AS TrafficKey,
  ARRAY_AGG(ShippingKey LIMIT 1)[OFFSET(0)] AS ShippingKey

FROM `ecommerce-dwh.ecommerce_staging.sales_cleaned_final`
GROUP BY InvoiceNo, StockCode;
```

Shipping Deduplication

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_shipping_dedup` AS
SELECT
  InvoiceNo,
  ANY_VALUE(ShippingKey) AS ShippingKey
FROM `ecommerce-dwh.ecommerce_dwh.dim_shipping`
GROUP BY InvoiceNo;
```

Web Traffic Deduplication

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_traffic_dedup` AS
SELECT
  Country,
  ANY_VALUE(TrafficKey) AS TrafficKey
FROM `ecommerce-dwh.ecommerce_dwh.dim_traffic`
GROUP BY Country;
```

Null Check

Null Key Check | Run | Save query | Download | Share

```sql
SELECT
  COUNTIF(CustomerKey IS NULL) AS null_customer_keys,
  COUNTIF(ProductKey IS NULL) AS null_product_keys,
  COUNTIF(HolidayKey IS NULL) AS null_holiday_keys
FROM `ecommerce-dwh.ecommerce_dwh.fact_sales`;
```

Revenue Check

Revenue Sanity Check | Run | Save query

```sql
-- From staging (fact_stage2)
SELECT ROUND(SUM(TotalAmount), 2) AS total_revenue_staging
FROM `ecommerce-dwh.ecommerce_staging.fact_stage2`;

-- From final fact
SELECT ROUND(SUM(TotalAmount), 2) AS total_revenue_fact
FROM `ecommerce-dwh.ecommerce_dwh.fact_sales`;
```

Removing Negative sales, Cancelled invoices

clean sales | Run | Save query | Download | Share

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_staging.sales_cleaned` AS
SELECT
  CAST(InvoiceNo AS STRING) AS InvoiceNo,
  StockCode,
  Description,
  CAST(Quantity AS INT64) AS Quantity,
  CAST(UnitPrice AS FLOAT64) AS UnitPrice,
  SAFE_CAST(CustomerID AS STRING) AS CustomerID,
  DATE(InvoiceDate) AS InvoiceDate,
  Country,
  (CAST(Quantity AS INT64) * CAST(UnitPrice AS FLOAT64)) AS TotalAmount
FROM `ecommerce-dwh.ecommerce_raw.raw_sales`
WHERE Quantity > 0
  AND UnitPrice > 0
  AND NOT STARTS_WITH(CAST(InvoiceNo AS STRING), 'C');
```

Load:

DimCustomer Run Save query Download

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_customer` AS
SELECT
  ROW_NUMBER() OVER() AS CustomerKey,
  CustomerID,
  CustomerName,
  Email,
  Gender,
  LoyaltyLevel,
  SignupDate,
  City,
  Country
FROM `ecommerce-dwh.ecommerce_raw.raw_customers_enriched`;
```

DimProduct Run Save query Download

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_product` AS
SELECT
  ROW_NUMBER() OVER() AS ProductKey,
  StockCode,
  Description,
  UnitPrice,
  Category,
  Supplier,
  Brand,
  LaunchDate,
  ReorderLevel
FROM `ecommerce-dwh.ecommerce_raw.raw_products_enriched`;
```

DimShipping Run Save query Download

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_shipping` AS
SELECT
  ROW_NUMBER() OVER() AS ShippingKey,
  InvoiceNo,
  ShipmentDate,
  DeliveryDate,
  Courier,
  ShippingCost,
  Status
FROM `ecommerce-dwh.ecommerce_raw.shipping_data`;
```

DimExchangeRate Run Save query Download

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_exchange_rate` AS
SELECT
  ROW_NUMBER() OVER() AS ExchangeKey,
  Country,
  Currency,
  Exchange_GBP
FROM `ecommerce-dwh.ecommerce_raw.exchange_rates`;
```

DimFeedback

Run | Save query | Download | Sh

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_feedback` AS
SELECT
  ROW_NUMBER() OVER() AS FeedbackKey,
  CustomerID,
  Rating,
  Review
FROM `ecommerce-dwh.ecommerce_raw.customer_feedback`;
```

DimHoliday

Run | Save query | Download | Sha

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_holiday` AS
SELECT
  ROW_NUMBER() OVER() AS HolidayKey,
  HolidayName,
  Date,
  Description
FROM `ecommerce-dwh.ecommerce_raw.holiday_calendar`;
```

DimTraffic

Run | Save query | Download | Share

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.dim_traffic` AS
SELECT
  ROW_NUMBER() OVER() AS TrafficKey,
  Date,
  Country,
  PageViews,
  UniqueVisitors,
  AvgSessionDuration
FROM `ecommerce-dwh.ecommerce_raw.website_traffic`;
```

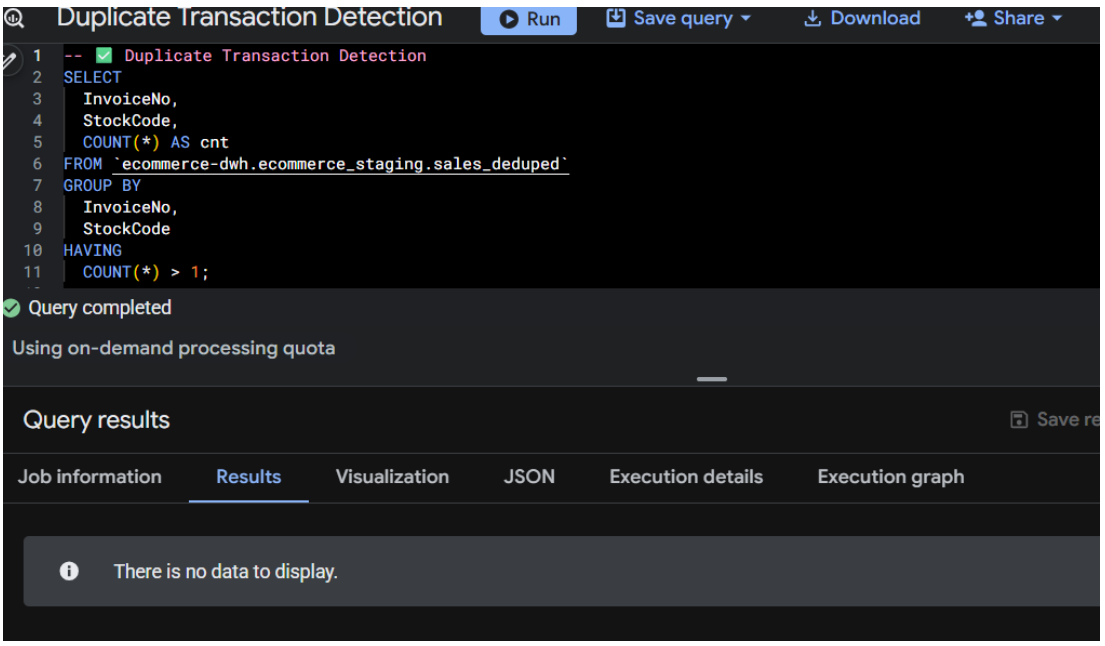
final clean Fact table creation

Run | Save query | Dow

```sql
CREATE OR REPLACE TABLE `ecommerce-dwh.ecommerce_dwh.fact_sales` AS
SELECT
  GENERATE_UUID() AS FactKey,
  InvoiceNo,
  CustomerKey,
  ProductKey,
  HolidayKey,
  ExchangeKey,
  TrafficKey,
  ShippingKey,
  Quantity,
  UnitPrice,
  TotalAmount,
  Country
FROM `ecommerce-dwh.ecommerce_staging.sales_deduped`;
```

7. Data Quality and Validation

1. **Schema Validation:**
    - Ensures that all columns adhere to the defined data types and naming conventions in the warehouse schema (e.g., `CustomerKey`, `ProductKey`, `SalesAmount`).
    - Verifies that mandatory columns such as `CustomerKey`, `ProductKey`, and `DateKey` are not null or mismatched.
2. **Referential Integrity Checks:**
    - Confirms that all foreign key relationships are properly maintained — for example, each `CustomerKey` in the `fact_sales` table exists in the `dim_customer` table.
    - Detects and flags orphan records, such as transactions referencing non-existent customers or products.
3. **Uniqueness and Duplication Checks:**
    - Ensures that key identifiers (like `CustomerKey`, `ProductKey`, `InvoiceNo`) remain unique across datasets.
    - Identifies and removes duplicate sales transactions or customer entries resulting from repeated data ingestion or CSV merges.
4. **Value Range and Consistency Checks:**
    - Validates that numerical fields (e.g., `Quantity`, `UnitPrice`, `TotalAmount`) fall within realistic and business-defined ranges.
    - Ensures consistency in categorical data, such as standardized country names, payment methods, and product categories.
5. **Null and Missing Data Handling:**
    - Detects null or missing values in critical dimensions or facts and applies predefined rules such as replacement with defaults or exclusion.
    - Ensures that incomplete transactional records are either corrected or excluded from analytics.



null check in fact and all dim

Run Schedule Open in

```sql
-- Check total rows and NULL ID counts across all fact & dimension tables
SELECT 'fact_sales' AS table_name,
       COUNT(*) AS total_rows,
       COUNTIF(FactKey IS NULL) AS null_ids
FROM `ecommerce-dwh.ecommerce_dwh.fact_sales`

UNION ALL
SELECT 'dim_customer' AS table_name,
       COUNT(*) AS total_rows,
       COUNTIF(CustomerKey IS NULL) AS null_ids
FROM `ecommerce-dwh.ecommerce_dwh.dim_customer`

UNION ALL
SELECT 'dim_exchange_rate' AS table_name,
       COUNT(*) AS total_rows,
       COUNTIF(ExchangeKey IS NULL) AS null_ids
FROM `ecommerce-dwh.ecommerce_dwh.dim_exchange_rate`

UNION ALL
SELECT 'dim_feedback' AS table_name,
       COUNT(*) AS total_rows,
       COUNTIF(FeedbackKey IS NULL) AS null_ids
FROM `ecommerce-dwh.ecommerce_dwh.dim_feedback`
```

```sql
UNION ALL
SELECT 'dim_holiday' AS table_name,
       COUNT(*) AS total_rows,
       COUNTIF(HolidayKey IS NULL) AS null_ids
FROM `ecommerce-dwh.ecommerce_dwh.dim_holiday`

UNION ALL
SELECT 'dim_product' AS table_name,
       COUNT(*) AS total_rows,
       COUNTIF(ProductKey IS NULL) AS null_ids
FROM `ecommerce-dwh.ecommerce_dwh.dim_product`

UNION ALL
SELECT 'dim_shipping_dedup' AS table_name,
       COUNT(*) AS total_rows,
       COUNTIF(ShippingKey IS NULL) AS null_ids
FROM `ecommerce-dwh.ecommerce_dwh.dim_shipping_dedup`

UNION ALL
SELECT 'dim_traffic_dedup' AS table_name,
       COUNT(*) AS total_rows,
       COUNTIF(TrafficKey IS NULL) AS null_ids
FROM `ecommerce-dwh.ecommerce_dwh.dim_traffic_dedup`;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	table_name	total_rows	null_ids
1	fact_sales	517881	0
2	dim_customer	4360	0
3	dim_exchange_rate	36	0
4	dim_feedback	349	0
5	dim_holiday	18	0
6	dim_product	16282	0
7	dim_shipping_dedup	4984	0
8	dim_traffic_dedup	37	0

Shows that there is no duplicate or null data row.

8. Warehouse Deployment

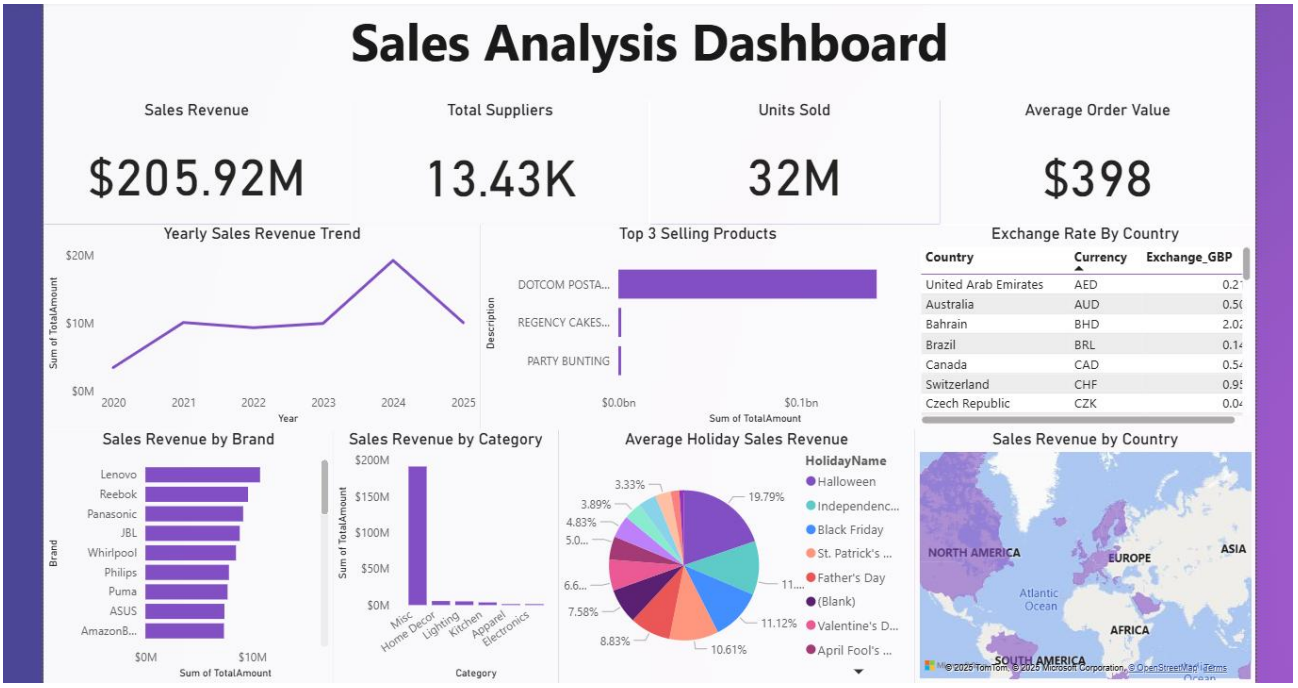
The Retail Customer Analytics Data Warehouse was deployed using **Google BigQuery** as the central platform for data storage, ETL processing, and query management. All data extraction, transformation, and loading activities were performed directly within BigQuery using SQL scripts, integrating multiple sources such as sales, customers, products, feedback, shipping, website traffic, and exchange rates into a unified star schema.

The finalized warehouse was connected to **Power BI** for KPI visualization and dashboard reporting, enabling interactive insights on sales performance, customer behavior, and product trends. Additionally, **Google Colab** was used for analytics and machine learning, leveraging BigQuery's connectors to access data directly for clustering, anomaly detection, and sentiment analysis.

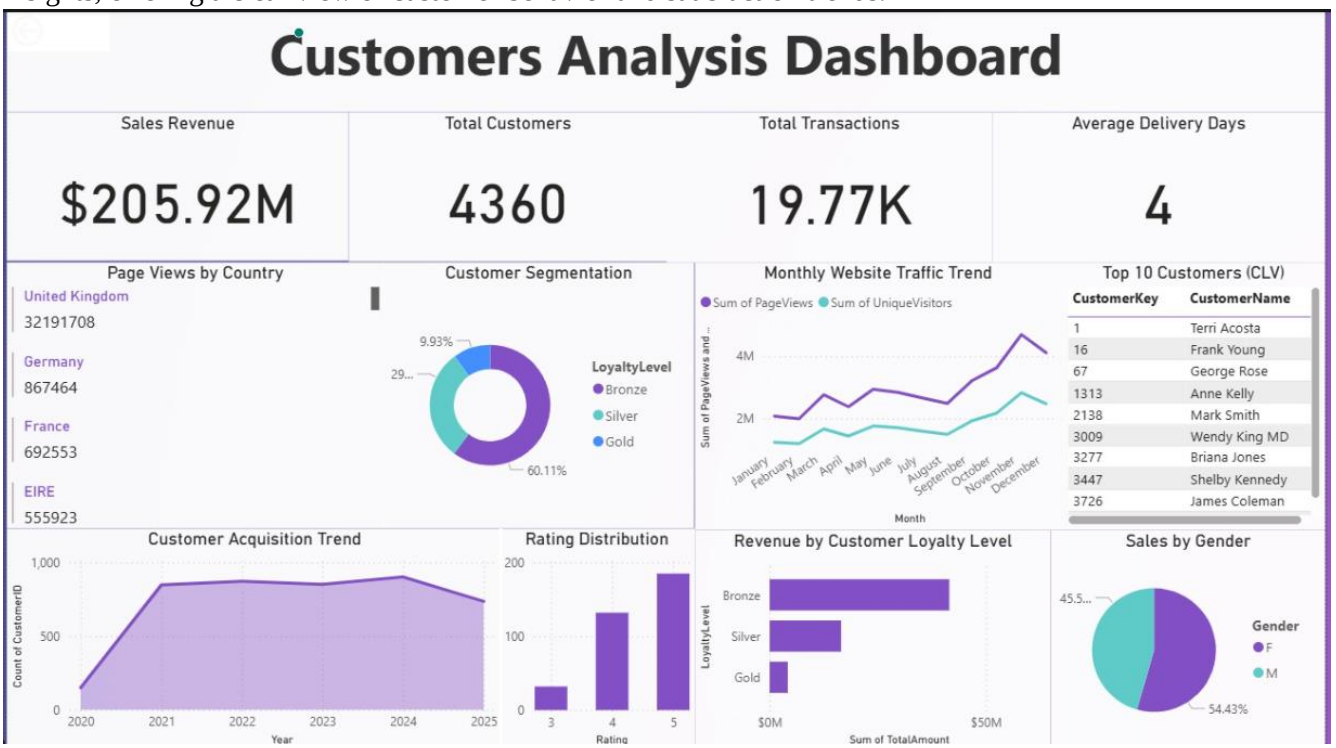
This integrated deployment ensures a seamless workflow where **BigQuery handles data management**, **Power BI delivers business intelligence**, and **Colab supports AI-driven insights**, resulting in a scalable, automated, and insight-rich retail analytics ecosystem.

9.1 BI Dashboard

The **Sales Analysis Dashboard** highlights overall business performance through KPIs such as total sales, suppliers, and units sold. It visualizes yearly revenue trends, top-selling products, and sales by brand and category, helping identify profitable areas and seasonal patterns.



The **Customer Analysis Dashboard** focuses on customer engagement with metrics like total customers, transactions, and average delivery time. It presents loyalty levels, ratings, and demographic insights, offering a clear view of customer behavior and satisfaction trends.

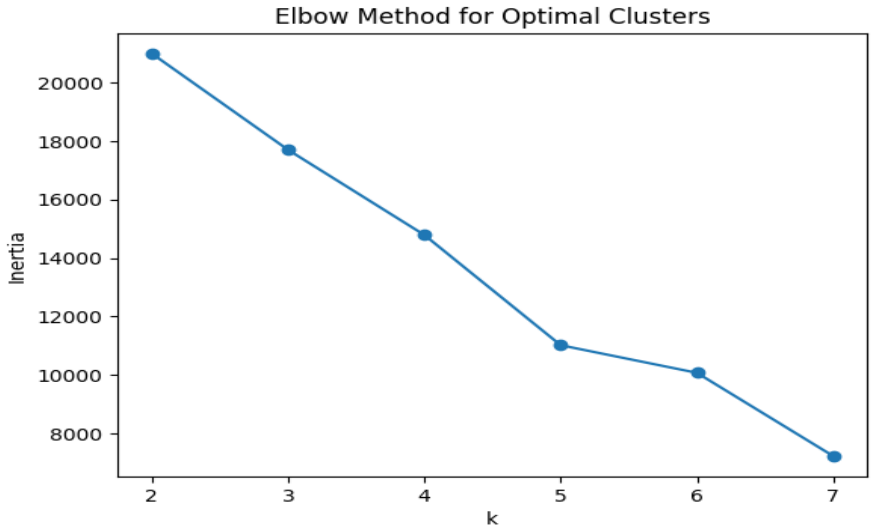


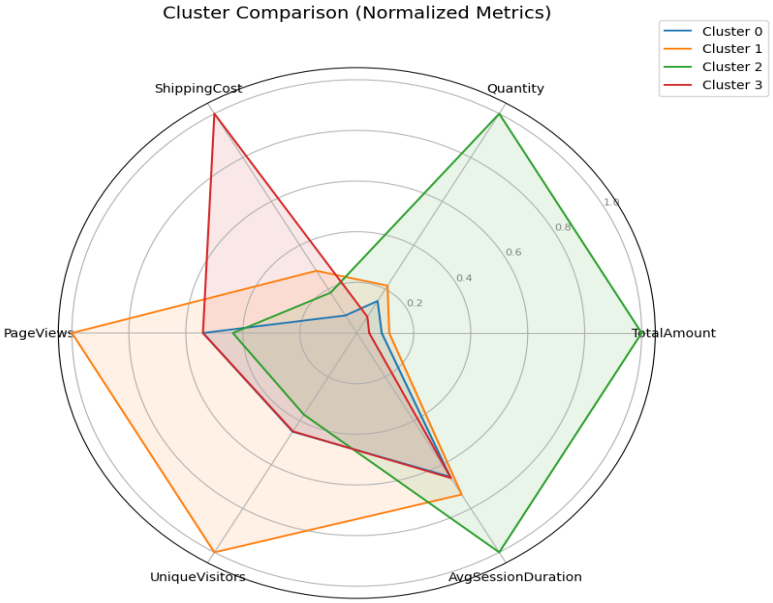
9.2 Analytics & ML Insights

9.2.1 K-Means Clustering (Customer Segmentation)

- **Technique Type:** Data Mining / Machine Learning
- **Purpose:** To segment customers into meaningful groups based on purchasing behavior, spending frequency, and total transaction value.
- **Tools Used:** *scikit-learn*

The **K-Means clustering algorithm** was applied to partition customers into clusters representing distinct behavior profiles such as *high-value*, *occasional*, and *low-engagement* shoppers. This segmentation enables targeted marketing strategies, personalized promotions, and better inventory planning. The model computes centroids for each cluster and assigns customers based on their proximity in feature space (e.g., total spending, number of purchases, and recency).

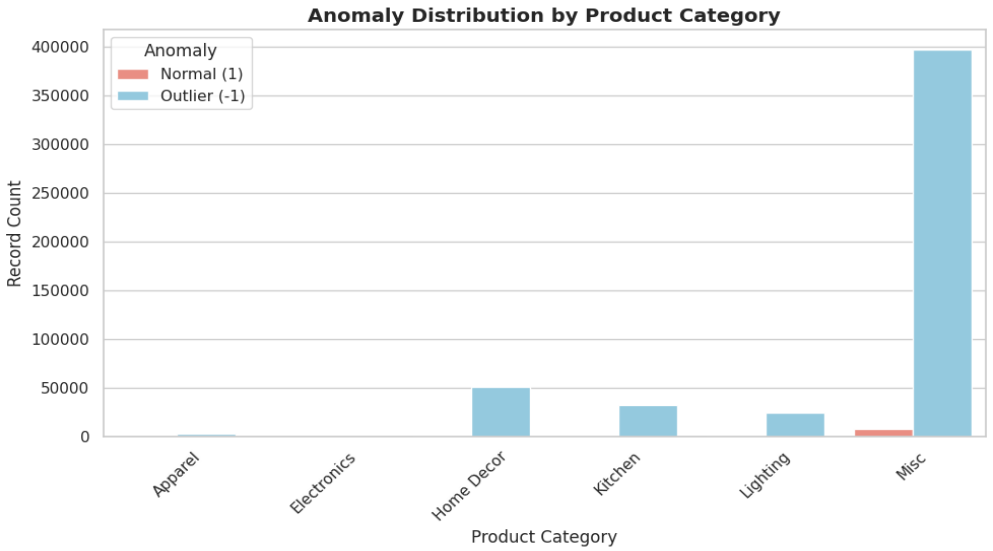


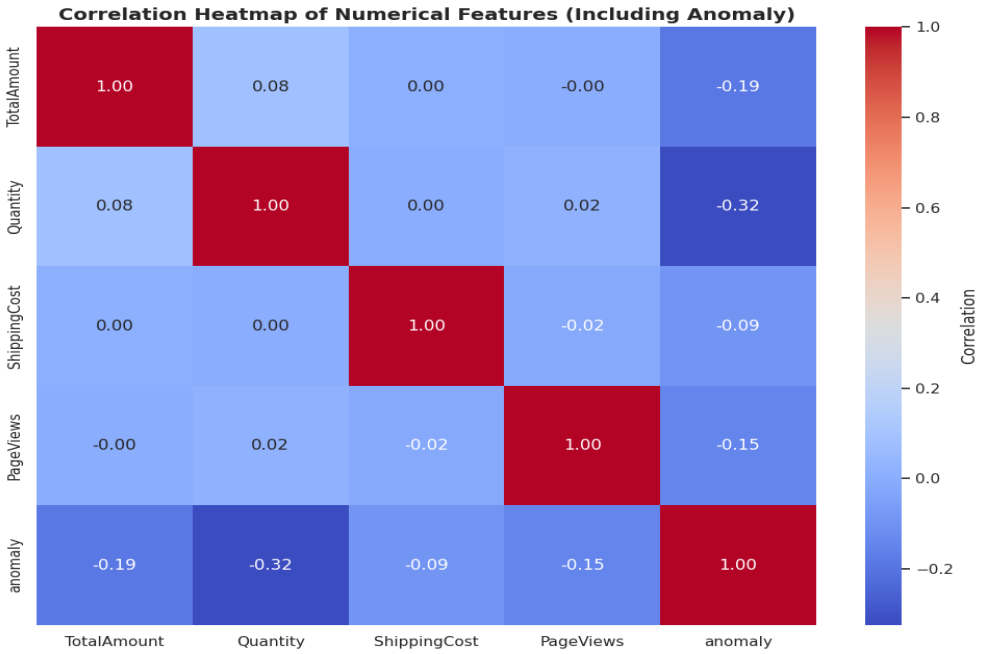


9.2.2 Isolation Forest (Sales Anomaly Detection)

- **Technique Type:** Machine Learning
- **Purpose:** To identify unusual or suspicious sales transactions that may indicate data entry errors, fraudulent activity, or abnormal purchasing trends.
- **Tools Used:** *scikit-learn*

The **Isolation Forest algorithm** was used to detect anomalies within sales data by isolating points that significantly deviate from normal transactional patterns. This unsupervised model builds random decision trees to isolate observations and assigns an anomaly score to each record. Transactions with high anomaly scores were flagged for further investigation, improving data quality and supporting fraud detection efforts.

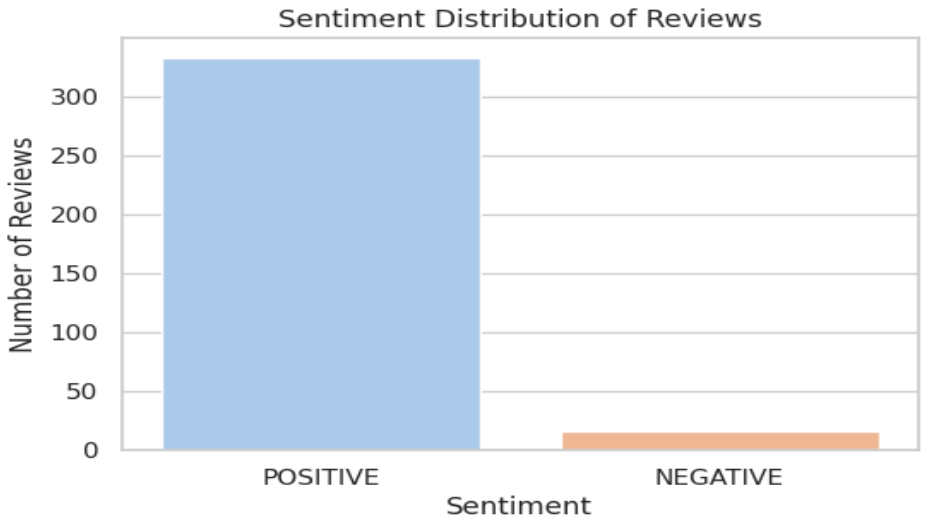


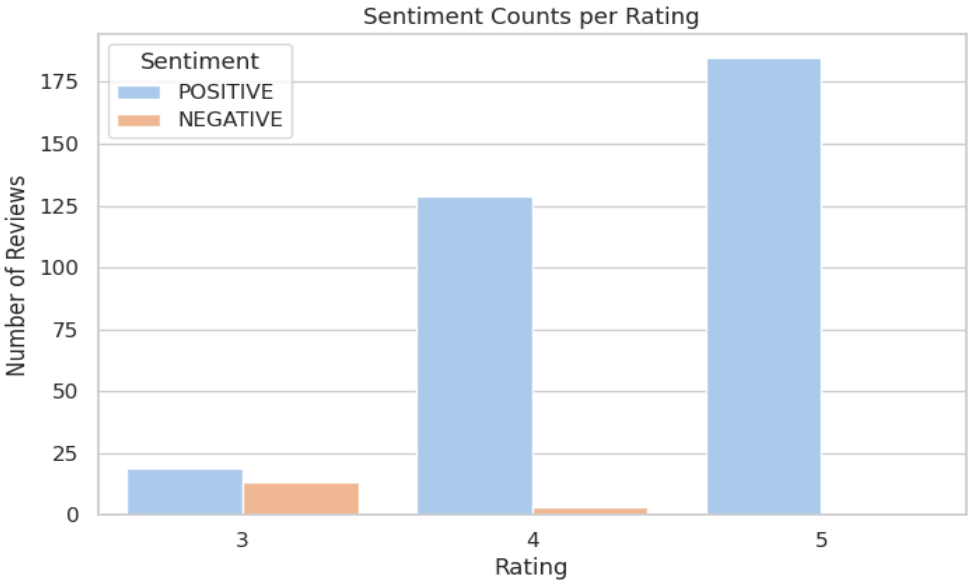


9.2.3 Sentiment Analysis (Customer Feedback Interpretation)

- **Technique Type:** Deep Learning / Natural Language Processing (NLP)
- **Purpose:** To analyze customer feedback and extract insights about overall satisfaction, product perception, and service quality.
- **Tools Used:** *Transformers (Hugging Face)*

A **Transformer-based sentiment analysis model** was employed to classify textual customer feedback into positive, neutral, and negative sentiments. Using pretrained deep learning models, such as BERT, the system interprets customer opinions to identify areas for improvement and track satisfaction trends over time. These insights enable the business to make data-driven decisions in product development, service enhancement, and customer experience optimization.





10. Conclusion

The Retail Customer Analytics Data Warehouse successfully integrates diverse data sources into a unified, scalable environment for comprehensive retail analysis. By leveraging **Google BigQuery** for ETL and storage, **Power BI** for KPI visualization, and **Google Colab** for advanced analytics and machine learning, the system delivers end-to-end data intelligence. It enables data-driven decision-making through actionable insights such as customer segmentation, sales anomaly detection, and sentiment evaluation. Overall, the project demonstrates how cloud-based data warehousing combined with AI and visualization tools can enhance business performance, operational efficiency, and customer understanding in the retail domain.